

AWS Solutions Architect - Associate

Graduation Project Ideas

Author: Ayman Aly Mahmoud

ayman@manara.tech

[Ayman Mahmoud | LinkedIn](#)

Project Deliverables

Learners are expected to submit the following:

1. Solution Architecture Diagram

- Create a visual representation of the solution architecture.
- Tools such as [Lucidchart](#), [draw.io](#) or any other free diagramming tools may be used.

2. GitHub Repository

- A public repository containing the complete project documentation (Please include the solution architecture diagram and the documentation in the README file).
- [Here](#) is an example for structure and content guidelines.

3. Optional Deliverable

- A live URL or a recorded video demonstrating the deployed solution on AWS (optional but encouraged).

Here is a list of 8 project ideas that learners can use for their graduation project.

Project 1: Scalable Web Application with ALB and Auto Scaling

Architecture: EC2-Based

Description: Deploy a production-grade web application on AWS using EC2 instances inside a properly architected VPC with public and private subnets across two Availability Zones. Achieve high availability and scalability with ALB, ASG, and a CloudFront distribution for caching static assets. A Multi-AZ RDS instance serves as the database backend, with all compute in private subnets.

Key AWS Services:

- **VPC:** Public & private subnets, NAT Gateway, Security Groups, NACLs
- **EC2 + ASG:** Launch Template, scaling policies (target tracking)
- **ALB + WAF:** Layer 7 routing, WAF rules for OWASP Top 10
- **CloudFront:** Cache static assets, reduce latency
- **RDS Multi-AZ:** MySQL/PostgreSQL with automated failover
- **Route 53:** Alias record pointing to ALB, health checks
- **Systems Manager:** Session Manager for secure instance access
- **CloudWatch + SNS:** Dashboards, alarms, and notifications

Learning Outcomes:

- Design VPCs with correct subnet, route table, and NAT Gateway configurations
- Build highly available architectures across multiple Availability Zones
- Configure ALB listener rules and target group health checks
- Implement Auto Scaling with target tracking and step scaling policies
- Secure applications with WAF, Security Groups, and private subnets
- Use Systems Manager Session Manager as a bastion-free access alternative

Project 2: Serverless Image Processing Pipeline with S3, SQS & Lambda

Architecture: Serverless

Description: Build a serverless image processing pipeline where uploads to an S3 source bucket publish events to an SQS queue. A Lambda function polls the queue, resizes and

watermarks images, then stores results in a destination S3 bucket. Step Functions orchestrate multi-step workflows (e.g., thumbnail + full-res + metadata extraction). CloudFront serves the processed images globally.

Key AWS Services:

- **S3 (source & destination):** Bucket policies, lifecycle rules, event notifications
- **SQS + DLQ:** Decouples events from processing; dead-letter queue for failures
- **AWS Lambda:** Image resize/watermark; Lambda Layers for Pillow/Sharp
- **Step Functions:** Standard workflow — validate → resize → watermark → store
- **API Gateway:** Pre-signed URL generation endpoint for uploads
- **DynamoDB:** Image metadata store (upload time, dimensions, status)
- **CloudFront:** Serve processed images with low latency globally
- **SNS:** Notify on job completion or failure

Learning Outcomes:

- Design event-driven architectures using S3 event notifications and SQS
 - Understand why SQS decoupling improves resilience and enables retries
 - Use Lambda Layers to package large dependencies (image libraries)
 - Orchestrate multi-step serverless workflows with Step Functions
 - Apply S3 lifecycle policies to transition or expire objects by storage class
 - Serve processed content through CloudFront with appropriate cache behaviors
-

Project 3: Serverless REST API with Cognito Auth, DynamoDB & WAF

Architecture: Serverless

Description: Build a serverless REST API for a to-do or customer records application using API Gateway, Lambda, and DynamoDB. Add Amazon Cognito User Pools for authentication, WAF for rate limiting and bot protection, X-Ray for distributed tracing, and API Gateway caching to reduce backend load. A CloudFront distribution sits in front of API Gateway for global edge delivery.

Key AWS Services:

- **API Gateway:** REST API, Cognito JWT authorizer, usage plans, response caching
- **Amazon Cognito:** User Pool for sign-up/sign-in, hosted UI, JWT tokens
- **AWS Lambda:** CRUD handler functions, environment variables, IAM roles
- **DynamoDB:** Table design with GSIs, on-demand capacity, DynamoDB Streams
- **WAF:** Rate-based rules, geo blocking, OWASP managed rule set
- **CloudFront:** CDN in front of API Gateway for global edge caching
- **X-Ray:** End-to-end distributed tracing across API GW → Lambda → DynamoDB
- **S3 + CloudFront:** Host and serve the static React/Vue frontend

Learning Outcomes:

- Implement token-based authentication with Cognito and API Gateway authorizers
 - Design DynamoDB table schemas and GSIs for efficient access patterns
 - Apply WAF rules to protect APIs from abuse and injection attacks
 - Use X-Ray to trace and debug latency across a serverless call chain
 - Enable API Gateway caching to reduce Lambda invocations and cut cost
 - Understand DynamoDB Streams for triggering downstream event processing
-

Project 4: Hybrid Cloud Connectivity with Transit Gateway and Site-to-Site VPN

Architecture: Hybrid / Network

Description: Design and implement a hybrid network architecture that connects an on-premises data center to AWS. Use an AWS Transit Gateway as the central network hub, attach multiple VPCs (Dev, Staging, Prod), and establish a Site-to-Site VPN connection. Configure Route 53 Resolver for split-horizon DNS so on-premises hosts can resolve private AWS records and vice versa. Understand when to use Direct Connect instead of VPN.

Key AWS Services:

- **VPC (multiple):** Isolated VPCs per environment; CIDR planning to avoid overlap
- **Transit Gateway:** Central hub; route tables, VPC attachments, VPN attachment
- **Site-to-Site VPN:** IKEv2 tunnel to on-premises; BGP routing with ASN configuration

- **AWS Direct Connect:** Dedicated link as VPN alternative; understand use case tradeoffs
- **Route 53 Resolver:** Inbound/outbound endpoints for hybrid DNS resolution
- **AWS RAM:** Share Transit Gateway with multiple AWS accounts
- **Network Firewall:** Centralized inspection for inter-VPC and egress traffic
- **CloudTrail + Config:** Audit network configuration changes

Learning Outcomes:

- Architect multi-VPC networks using Transit Gateway as a hub-and-spoke model
 - Configure Site-to-Site VPN with BGP and understand static vs dynamic routing
 - Compare Direct Connect vs Site-to-Site VPN for bandwidth, cost, and reliability
 - Set up Route 53 Resolver endpoints for split-horizon DNS in hybrid environments
 - Use AWS RAM to share networking resources across accounts and organizations
 - Apply Network Firewall for centralized traffic inspection and east-west filtering
-

Project 5: Multi-Region Disaster Recovery with Route 53 Failover

Architecture: Hybrid / DR

Description: Implement a multi-region disaster recovery strategy for a web application. The primary region (us-east-1) runs the full stack. The secondary region (eu-west-1) maintains warm standby resources. Route 53 health checks detect primary region failure and automatically fail over DNS. Learners study and compare all four DR strategies — Backup & Restore, Pilot Light, Warm Standby, and Multi-Site Active-Active — and implement the Warm Standby pattern.

Key AWS Services:

- **Route 53:** Failover routing policy, health checks, DNS TTL tuning
- **Aurora Global Database:** Primary cluster + read-only replica in secondary region (sub-1s RPO)
- **S3 Cross-Region Replication:** Replicate user data and static assets to secondary region
- **DynamoDB Global Tables:** Multi-region active-active replication for low-latency reads

- **EC2 + ASG (both regions):** Warm standby — minimal instances, scale up on failover trigger
- **CloudFormation StackSets:** Deploy identical infrastructure across regions consistently
- **AWS Backup:** Centralized backup plans for RDS, EC2, and S3
- **CloudWatch + EventBridge:** Cross-region alarms; trigger failover automation via Lambda

Learning Outcomes:

- Understand and compare the four AWS DR strategies by RTO, RPO, and cost
 - Configure Route 53 failover routing with health checks and DNS TTL considerations
 - Use Aurora Global Database for near-zero RPO cross-region database replication
 - Implement S3 Cross-Region Replication with replication rules and IAM roles
 - Deploy multi-region infrastructure consistently using CloudFormation StackSets
 - Design recovery automation using EventBridge rules and Lambda-driven runbooks
-

Project 6: Containerized Microservices with ECS Fargate and Service Discovery

Architecture: Containers

Description: Migrate a monolithic Node.js application into three microservices — Auth, Orders, and Notifications — running on Amazon ECS Fargate. Services communicate via AWS Cloud Map for service discovery and an Application Load Balancer for external traffic. Secrets are stored in AWS Secrets Manager and injected at runtime. Use CodePipeline and CodeDeploy for blue/green container deployments. ElastiCache Redis handles shared session caching across stateless container instances.

Key AWS Services:

- **ECS Fargate:** Task definitions, services, capacity providers; no server management
- **ECR:** Private container registry with image vulnerability scanning on push
- **ALB + Target Groups:** Route traffic to multiple ECS services by path (e.g. /api/orders)

- **AWS Cloud Map:** Service discovery so containers find each other by DNS name
- **Secrets Manager:** Inject database credentials and API keys into containers at runtime
- **ElastiCache (Redis):** Shared session store across stateless container instances
- **CodePipeline + CodeDeploy:** CI/CD pipeline with blue/green deployment and automatic rollback
- **X-Ray:** Distributed tracing across microservices with service map visualization

Learning Outcomes:

- Build and push Docker images to ECR and configure ECS task definitions
 - Design ECS Fargate services with correct IAM task roles and execution roles
 - Implement service-to-service communication using Cloud Map DNS-based discovery
 - Configure ALB path-based routing rules to front multiple microservices
 - Set up blue/green deployments using CodeDeploy with ECS integration
 - Manage secrets securely with Secrets Manager and avoid hardcoded credentials
-

Project 7: Serverless Data Lake and Analytics Pipeline

Architecture: Data & Analytics

Description: Build a fully serverless data lake on Amazon S3 for a retail use case. Raw transactional data lands in S3 via Kinesis Data Firehose. AWS Glue crawlers infer schemas and populate the Data Catalog. Glue ETL jobs transform raw JSON into partitioned Parquet format. Amazon Athena queries the transformed data with zero infrastructure. Amazon QuickSight visualizes KPIs in an executive dashboard with SPICE caching.

Key AWS Services:

- **S3 (data lake):** Raw, curated, and aggregated zones; Intelligent Tiering; lifecycle policies
- **Kinesis Data Firehose:** Real-time ingestion with buffering, compression, and S3 delivery
- **AWS Glue:** Crawlers for schema discovery; Spark ETL jobs; centralized Data Catalog

- **Amazon Athena:** Serverless SQL queries on S3; partitioning and bucketing for cost control
- **AWS Lake Formation:** Fine-grained column-level and row-level access control on the data lake
- **Amazon QuickSight:** SPICE-powered dashboards; row-level security for per-user data access
- **EventBridge Scheduler:** Trigger Glue jobs on schedule; orchestrate pipeline steps
- **IAM + KMS:** Encryption at rest; lake-level and table-level access policies

Learning Outcomes:

- Design a three-zone (raw / curated / aggregated) data lake architecture on S3
- Ingest streaming data into S3 using Kinesis Data Firehose with inline transformation
- Use Glue crawlers and ETL jobs to convert JSON into partitioned Parquet format
- Write Athena SQL with partitioning filters to minimize data scanned and reduce cost
- Apply Lake Formation to enforce column-level and row-level data governance
- Build QuickSight dashboards connected to Athena with SPICE in-memory caching

Project 8: Secure Multi-Tier Architecture with GuardDuty, KMS & Security Hub

Architecture: Security

Description: Design and implement a defense-in-depth security architecture for a three-tier application (web, app, data). Apply encryption at rest and in transit, centralized secret management, automated threat detection, compliance auditing, and incident response automation using EventBridge and Lambda. This project maps directly to the Security domain of the SAA-C03 exam and the AWS Well-Architected Framework Security Pillar.

Key AWS Services:

- **AWS KMS:** Customer-managed keys (CMKs) for encrypting S3, RDS, EBS, and SSM Parameter Store values

- **Secrets Manager:** Automatic credential rotation for RDS; full audit log of every secret access
- **GuardDuty:** Threat detection — unusual API calls, crypto-mining activity, reconnaissance
- **Security Hub:** Aggregated security findings from all services; CIS AWS Benchmark compliance score
- **AWS Config:** Detect and auto-remediate non-compliant resources using Config Rules and SSM documents
- **CloudTrail:** Full API audit log across all regions; log file integrity validation enabled
- **WAF + Shield Standard:** Edge protection against SQLi, XSS, and volumetric DDoS attacks
- **IAM + SCPs:** Least-privilege roles, permission boundaries, deny-root-usage Service Control Policies

Learning Outcomes:

- Apply KMS customer-managed keys with key policies and grant-based access controls
- Configure Secrets Manager automatic rotation for RDS using a Lambda rotation function
- Enable GuardDuty and interpret finding types (Recon, Backdoor, Cryptocurrency)
- Aggregate findings in Security Hub and map them to CIS and PCI DSS compliance frameworks
- Write Config Rules and attach SSM remediation documents for automatic non-compliance fixes
- Design IAM least-privilege roles using permission boundaries and organization-level SCPs